



**"RANKED 45
IN INDIA**

#University Category



**WORLD
UNIVERSITY
RANKINGS**

NO.1 PVT. UNIVERSITY IN
ACADEMIC REPUTATION IN INDIA



ACCREDITED GRADE 'A'
BY NAAC



PERFECT SCORE OF **150/150** AS A TESTAMENT
TO EXCEPTIONAL E-LEARNING METHODS

1



Applied Machine Learning

Unit 05 – Lecture 09: Maximum Likelihood and Over-fitting

Tofik Ali

School of Computer Science, UPES Dehradun

Autumn 2026

Direct regression, R^2 , residual diagnostics and cross-validation

Topic focus: **why least squares**

Repository: <https://github.com/tali7c/Applied-Machine-Learning>

Sources: James et al., *An Introduction to Statistical Learning with Python*, Ch. 3–6.; Bishop, *Pattern Recognition and Machine Learning*, Ch. 3.; Deisenroth, Faisal and Ong, *Mathematics for Machine Learning*, Ch. 9.; Müller and Guido, *Introduction to Machine Learning with Python*, §2.3.

Direct
oooo

MLE
oooooooooooo

R^2
oooooooooo

Adequacy
ooooooo

Over-fitting
oooooo

CV
ooooo

Summary
oo

Quick Links

[Direct regression](#)

[Maximum likelihood](#)

[R²](#)

[Model adequacy](#)

[Over-fitting](#)

[Cross-validation](#)

[Summary](#)

Agenda

The direct regression method

Maximum likelihood estimation

The coefficient of determination

Checking model adequacy

Over-fitting and its detection

Cross-validation as the detector

Summary

How this lecture works

Every idea appears twice

1. **By hand** — five points, three numbers, a 2×2 table. Small enough to check on paper, and the same five points are used from the first slide to the last.
2. **In code** — the same arithmetic on real data, so you see that `r2_score` and `cross_val_score` do exactly that and nothing more.

Commit before you look

Five slides ask you to predict a number before it is revealed. Write your guess down. Being wrong on paper is how the idea sticks.

The centrepiece is a proof: **least squares is not a convenient guess. It is the maximum likelihood estimate under a Gaussian noise assumption.** Everything after it is about noticing when that estimate has flattered you.

Where we are

Last lecture produced an estimator

Simple linear regression, the normal equations, and the closed form

$$b_1 = \frac{S_{xy}}{S_{xx}}, \quad b_0 = \bar{y} - b_1 \bar{x}.$$

We minimised $\sum (y_i - b_0 - b_1 x_i)^2$ because squares are smooth and differentiable.

Three questions that were left open

Why squares? Why not $|\cdot|$, or the perpendicular distance?

Is the fit any good? One number, R^2 — and what it hides.

Will it work tomorrow? A fit that describes today's rows perfectly can be useless on the next batch.

Today answers all three

And the answer to the first one — maximum likelihood — is the reason the other two are even askable.

Three kinds of seed, and why they are not the same

Lecture seed = 0

Every *arbitrary* draw: the 20 000 replications, the noise columns, the folds.
`default_rng(0),`
`random_state=0.`

Data seed = 3

Fixes the 25 points from $y = \sin 2\pi x + \varepsilon$ that the over-fitting half is about. It chooses the *data*, not the outcome, so it is named apart and never moved.

Split sweep 0 ... 9

Ten validation splits, run on purpose. **The disagreement between them is the result** — collapsing them to one seed would delete the demonstration.

Why this matters to you

Every number in this deck is printed by one script. Run the code as written and you get the same numbers — which is the only reason to believe a number on a slide.

“Closest” is a choice, not a fact

Fit a line to a cloud of points. Closest *in what sense*?

Method	Minimises	When it is right
Direct	$\sum (y_i - b_0 - b_1 x_i)^2$ — vertical distances	x is known exactly; you want to <i>predict</i> y from x
Inverse	$\sum (x_i - c_0 - c_1 y_i)^2$ — horizontal distances	you want to predict x from y
Orthogonal	$\sum d_i^2$ — perpendicular distances	both variables are measured with error

The direct method is the one this course means by “regression”

It treats x as fixed and known, and all the error as living in y . That asymmetry is a **modelling assumption**, and it is the assumption the next section turns into a probability statement.

By hand: five points, three answers

$$x = (1, 2, 3, 4, 5), \quad y = (2, 4, 5, 4, 5).$$

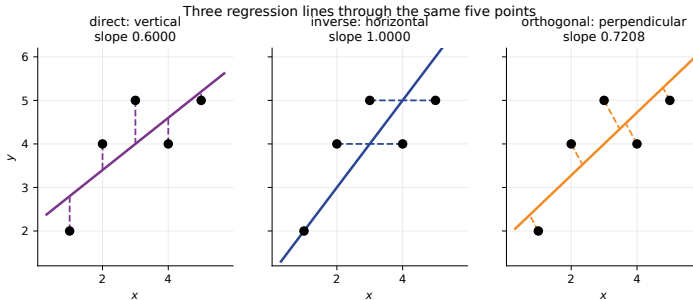
$$\bar{x} = 3, \bar{y} = 4, \text{ and } S_{xx} = 10, S_{yy} = 6, S_{xy} = 6.$$

Direct	$b_1 = S_{xy}/S_{xx} = 6/10$	0.6000
Inverse	fit x on y : $S_{xy}/S_{yy} = 1$, then invert: $S_{yy}/S_{xy} = 6/6$	1.0000
Orthogonal	top eigenvector of $\begin{pmatrix} 2.5 & 1.5 \\ 1.5 & 1.5 \end{pmatrix}$	0.7208

The two least-squares slopes bracket the truth, and their ratio is r^2

$\frac{b_1^{\text{direct}}}{b_1^{\text{inverse}}} = \frac{0.6}{1.0} = 0.6 = r^2$. They coincide only when $r^2 = 1$. **Three defensible lines through five points, and the data does not choose between them — your question does.**

Drawn: the three lines and the distances they minimise



The dashed segments are what each method squares and adds. Every line passes through $(\bar{x}, \bar{y}) = (3, 4)$ — that is forced by the intercept equation — and they differ only in how they pivot about it.

Each method wins its own contest, and only its own

```
for slope in (direct, inverse, orthogonal):  
    a = ybar - slope * xbar  
    r = y - (a + slope * x)  
    print((r**2).sum(), (r**2).sum() / (1 + slope**2), (r**2).sum() / slope  
          **2)
```

	vertical SS	perpendicular SS	horizontal SS
direct	2.4000	1.7647	6.6667
inverse	4.0000	2.0000	4.0000
orthogonal	2.5458	1.6754	4.9006

Each method wins its own contest, and only its own

```
for slope in (direct, inverse, orthogonal):  
    a = ybar - slope * xbar  
    r = y - (a + slope * x)  
    print((r**2).sum(), (r**2).sum() / (1 + slope**2), (r**2).sum() / slope  
          **2)
```

	vertical SS	perpendicular SS	horizontal SS
direct	2.4000	1.7647	6.6667
inverse	4.0000	2.0000	4.0000
orthogonal	2.5458	1.6754	4.9006

Read down the diagonal: 2.4000, 4.0000, 1.6754 — each is the smallest number in its own column and **nowhere else**. There is no “best line”; there is a best line *for a stated criterion*. **From here on, “regression” means the direct method**, and the next section says why that criterion in particular.

The question least squares never answered

Why the sum of *squares*?

The unsatisfying answers

“It is differentiable.” “It has a closed form.”
“It punishes big errors more.” All true, all true of $\sum e_i^4$ too.

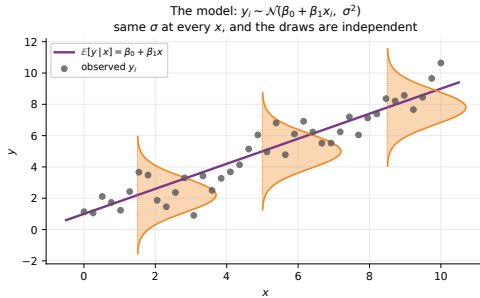
The real answer

Write down a **probability model** for how the data was generated. Ask which parameters make the data you actually observed most probable. **The answer is least squares** — provided the noise is Gaussian.

This cuts both ways, and that is the point

Least squares is *justified* under a Gaussian noise assumption — and *only* there. Change the noise model and the optimal estimator changes with it. We will measure that at the end of the section.

The probabilistic model, drawn



$y_i = \beta_0 + \beta_1 x_i + \varepsilon_i$, with $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$ independently. The line is the *mean* of y at each x ; the orange bells are the distribution around it. **Three assumptions in one picture: the mean is linear, the spread is the same everywhere, and the draws are independent.**

Step 1 — write the likelihood

Write $e_i = y_i - \beta_0 - \beta_1 x_i$. Each observation has density

$$p(y_i | x_i, \beta_0, \beta_1, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{e_i^2}{2\sigma^2}\right).$$

The draws are independent, so the joint density of all n of them is the product. Read as a function of the *parameters* with the data held fixed, that product is the **likelihood**:

$$L(\beta_0, \beta_1, \sigma^2) = \prod_{i=1}^n \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{e_i^2}{2\sigma^2}\right) = (2\pi\sigma^2)^{-n/2} \exp\left(-\frac{1}{2\sigma^2} \sum_{i=1}^n e_i^2\right).$$

A product of n exponentials of squares — already the sum of squares has appeared, inside the exponent, and we have not chosen anything yet.

Step 2 — take logarithms, and read the result off

log is strictly increasing, so it does not move the maximiser:

$$\ell(\beta_0, \beta_1, \sigma^2) = \log L = -\frac{n}{2} \log(2\pi) - \frac{n}{2} \log \sigma^2 - \underbrace{\frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2}_{S(\beta_0, \beta_1) = \text{SSE}}.$$

The whole derivation is now one sentence

Only the **last** term contains β_0 and β_1 , it carries a **minus** sign, and its multiplier $1/(2\sigma^2)$ is **positive**. So for *any* fixed $\sigma^2 > 0$:

$$\arg \max_{\beta_0, \beta_1} \ell = \arg \min_{\beta_0, \beta_1} S(\beta_0, \beta_1).$$

Maximising the Gaussian likelihood is minimising the sum of squared errors.

Setting $\partial S / \partial \beta_0 = \partial S / \partial \beta_1 = 0$ returns L8's normal equations unchanged. **The estimator did not move; its justification did.**

Predict first #1

Take the five points. Evaluate the log-likelihood on a grid of slopes, once with $\sigma^2 = 0.48$ and once with $\sigma^2 = 1$ — two *different* noise assumptions.

Do the two settings agree on the best slope? And does either agree with $S_{xy}/S_{xx} = 0.6$?

Predict first #1

Take the five points. Evaluate the log-likelihood on a grid of slopes, once with $\sigma^2 = 0.48$ and once with $\sigma^2 = 1$ — two *different* noise assumptions.

Do the two settings agree on the best slope? And does either agree with $S_{xy}/S_{xx} = 0.6$?

b1	SSE	logL(sigma^2=0.48)	logL(sigma^2=1)
0.40	2.8000	-5.676436	-5.994693
0.50	2.5000	-5.363936	-5.844693
0.60	2.4000	-5.259770	-5.794693
0.70	2.5000	-5.363936	-5.844693
0.80	2.8000	-5.676436	-5.994693
argmin SSE		b1 = 0.60000	
argmax logL s2=0.48		b1 = 0.60000	
argmax logL s2=1.00		b1 = 0.60000	

Predict first #1

Take the five points. Evaluate the log-likelihood on a grid of slopes, once with $\sigma^2 = 0.48$ and once with $\sigma^2 = 1$ — two *different* noise assumptions.

Do the two settings agree on the best slope? And does either agree with $S_{xy}/S_{xx} = 0.6$?

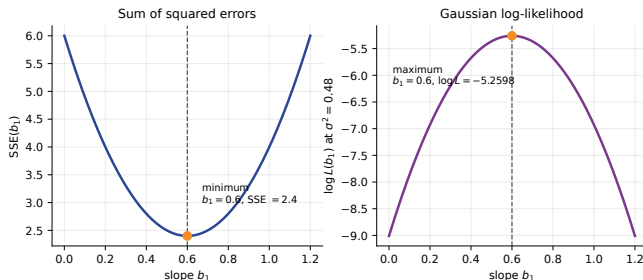
b1	SSE	logL(sigma^2=0.48)	logL(sigma^2=1)
0.40	2.8000	-5.676436	-5.994693
0.50	2.5000	-5.363936	-5.844693
0.60	2.4000	-5.259770	-5.794693
0.70	2.5000	-5.363936	-5.844693
0.80	2.8000	-5.676436	-5.994693
argmin SSE		b1 = 0.60000	
argmax logL s2=0.48		b1 = 0.60000	
argmax logL s2=1.00		b1 = 0.60000	

Same slope, to five decimal places, at both noise levels

The two log-likelihood columns are *different numbers* — -5.2598 against -5.7947 at the optimum — and they peak in exactly the same place. σ^2 **scales the likelihood; it cannot move the slope.** That is the algebra of the previous slide, measured.

Drawn: one curve upside down is the other

The same b_1 : maximising the likelihood IS minimising the SSE



Left, SSE as a function of the slope — a parabola with its minimum at $b_1 = 0.6$, $SSE = 2.4$.
Right, the Gaussian log-likelihood — the same parabola, negated and shifted, with its maximum at the same $b_1 = 0.6$. $\ell = \text{const} - SSE/(2\sigma^2)$, and that is the whole picture.

Step 3 — the MLE of σ^2

Now hold $\hat{\beta}_0, \hat{\beta}_1$ at their optimum and differentiate ℓ with respect to σ^2 , treating σ^2 as one symbol:

$$\frac{\partial \ell}{\partial \sigma^2} = -\frac{n}{2\sigma^2} + \frac{\text{SSE}}{2(\sigma^2)^2} = 0 \quad \Rightarrow \quad \boxed{\hat{\sigma}_{\text{ML}}^2 = \frac{\text{SSE}}{n}}$$

Substituting back gives the maximised log-likelihood in closed form,
 $\ell_{\text{max}} = -\frac{n}{2}(\log(2\pi\hat{\sigma}_{\text{ML}}^2) + 1)$.

But this one *is* wrong, in a way the slope was not

$\mathbb{E}[\text{SSE}] = (n-2)\sigma^2$ for a simple linear regression, because two parameters were spent fitting the line. Therefore

$$\mathbb{E}[\hat{\sigma}_{\text{ML}}^2] = \frac{n-2}{n}\sigma^2 < \sigma^2.$$

The maximum likelihood estimate of the noise variance is biased downwards.
The unbiased version divides by the degrees of freedom: $s^2 = \text{SSE}/(n-2)$.

The same five points: 0.48 against 0.80

```
sse = ((y - yhat)**2).sum()
print(sse / n, sse / (n - 2), np.sqrt(sse / (n - 2)))
```

```
SSE          2.4000
sigma^2 MLE   = SSE/n      = 2.4000/5 = 0.4800
sigma^2 unbias = SSE/(n-2) = 2.4000/3 = 0.8000
ratio (n-2)/n = 0.6000      s = sqrt(unbiased) = 0.8944
maximised log-likelihood = -n/2*(log(2*pi*s2_ml)+1) = -5.2598
direct evaluation                                     = -5.2598
```

The same five points: 0.48 against 0.80

```
sse = ((y - yhat)**2).sum()
print(sse / n, sse / (n - 2), np.sqrt(sse / (n - 2)))
```

```
SSE          2.4000
sigma^2 MLE   = SSE/n          = 2.4000/5 = 0.4800
sigma^2 unbias = SSE/(n-2)     = 2.4000/3 = 0.8000
ratio (n-2)/n = 0.6000         s = sqrt(unbiased) = 0.8944
maximised log-likelihood = -n/2*(log(2*pi*s2_ml)+1) = -5.2598
direct evaluation                                     = -5.2598
```

At $n = 5$ the MLE is **40%** too small in expectation, because $(n - 2)/n = 3/5$. **Two parameters out of five observations is a lot to spend.** At $n = 100$ the same factor is 0.98 and nobody notices — but the small-sample case is exactly the case in which you needed the estimate.

Predict first #2

Simulate 20,000 datasets of $n = 10$ points from $y = 1 + 2x + \varepsilon$ with $\sigma^2 = 4$ known and true. Fit each one and record SSE/n .

What does SSE/n average to across the 20,000 runs? And in what fraction of runs is it below 4?

Predict first #2

Simulate 20,000 datasets of $n = 10$ points from $y = 1 + 2x + \varepsilon$ with $\sigma^2 = 4$ known and true. Fit each one and record SSE/n .

What does SSE/n average to across the 20,000 runs? And in what fraction of runs is it below 4?

```
true sigma^2                4.0000  (n = 10, 20000 replications)
mean of SSE/n      (the MLE)  3.2117  bias -0.7883
mean of SSE/(n-2) (unbiased)  4.0146  bias +0.0146
predicted MLE mean (n-2)/n*s^2 3.2000
the MLE was too small in 73.4% of the 20000 replications
```

Predict first #2

Simulate 20,000 datasets of $n = 10$ points from $y = 1 + 2x + \varepsilon$ with $\sigma^2 = 4$ known and true. Fit each one and record SSE/n .

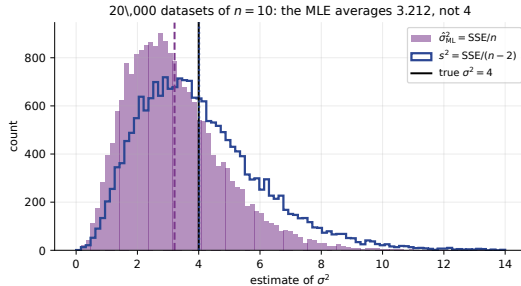
What does SSE/n average to across the 20,000 runs? And in what fraction of runs is it below 4?

```
true sigma^2                4.0000  (n = 10, 20000 replications)
mean of SSE/n      (the MLE)  3.2117  bias -0.7883
mean of SSE/(n-2) (unbiased)  4.0146  bias +0.0146
predicted MLE mean (n-2)/n*s^2 3.2000
the MLE was too small in 73.4% of the 20000 replications
```

3.2117 against a prediction of 3.2000

The theory said $(n-2)/n \times 4 = 0.8 \times 4 = 3.2$ and the simulation returned 3.2117. **This is not sampling noise; it is a systematic 20% understatement, and it happened in nearly three runs out of four.** A model that reports its own noise as smaller than it is will also report its prediction intervals as narrower than they are.

Drawn: two estimators, one truth



Both histograms are right-skewed — SSE is a scaled χ_{n-2}^2 — and they are the *same* histogram, stretched by $n/(n-2) = 1.25$. The purple mean sits at 3.21, the blue at 4.01, the black line at the truth. **Unbiased does not mean accurate: both estimators are wildly variable at $n = 10$.**

Change the noise model and the estimator changes

60 points, $y = 2 + 1.5x + \varepsilon$, ε **Laplace**, plus one heavy draw. For Laplace noise the log-likelihood contains $\sum |e_i|$, so the MLE is **least absolute deviations**, not least squares.

```
true intercept 2.0000 true slope 1.5000
Gaussian MLE = least squares : intercept 3.0024 slope 1.3738
Laplace MLE = least abs. dev. : intercept 2.7650 slope 1.3730
sum of squared residuals OLS 622.2965 LAD 625.7886
sum of |residuals| OLS 85.8123 LAD 83.6585
```

Change the noise model and the estimator changes

60 points, $y = 2 + 1.5x + \varepsilon$, ε **Laplace**, plus one heavy draw. For Laplace noise the log-likelihood contains $\sum |e_i|$, so the MLE is **least absolute deviations**, not least squares.

```
true intercept 2.0000 true slope 1.5000
Gaussian MLE = least squares      : intercept 3.0024 slope 1.3738
Laplace MLE = least abs. dev.    : intercept 2.7650 slope 1.3730
sum of squared residuals OLS 622.2965 LAD 625.7886
sum of |residuals| OLS 85.8123 LAD 83.6585
```

Each estimator wins its own criterion and loses the other

OLS has the smaller $\sum e_i^2$ ($622.30 < 625.79$); LAD has the smaller $\sum |e_i|$ ($83.66 < 85.81$). **Neither is cheating.** The LAD intercept is 2.77 against a truth of 2.00; the OLS intercept is 3.00. The two slopes are, on this one draw, the same to three decimals — **sixty points cannot show an efficiency gap, and we will not pretend they do.** What they do show: **the criterion you minimise follows from the noise distribution you assume.**

What R^2 actually compares

How much better is my line than the flat line $\hat{y} = \bar{y}$?

$$\underbrace{\sum (y_i - \bar{y})^2}_{SS_{\text{tot}}} = \underbrace{\sum (\hat{y}_i - \bar{y})^2}_{SS_{\text{reg}}} + \underbrace{\sum (y_i - \hat{y}_i)^2}_{SS_{\text{res}}}, \quad R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}} = \frac{SS_{\text{reg}}}{SS_{\text{tot}}}.$$

Read the two baselines

$R^2 = 0$: the line is no better than the mean.

$R^2 = 1$: every residual is zero.

The decomposition needs the intercept

$SS_{\text{tot}} = SS_{\text{reg}} + SS_{\text{res}}$ holds because $\sum e_i = 0$ and $\sum \hat{y}_i e_i = 0$, and both come from the normal equations. **Drop the intercept and the identity — and the interpretation — collapse.**

By hand: the whole calculation on five rows

x	y	$\hat{y} = 2.2 + 0.6x$	$y - \bar{y}$	$(y - \bar{y})^2$	$y - \hat{y}$	$(y - \hat{y})^2$
1	2	2.8	-2.0	4.00	-0.8	0.64
2	4	3.4	0.0	0.00	0.6	0.36
3	5	4.0	1.0	1.00	1.0	1.00
4	4	4.6	0.0	0.00	-0.6	0.36
5	5	5.2	1.0	1.00	-0.2	0.04
				$SS_{\text{tot}} = 6.00$	$SS_{\text{res}} = 2.40$	

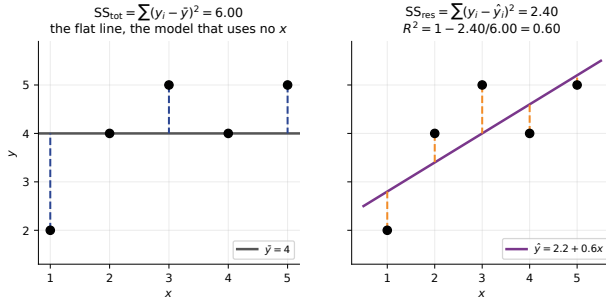
$$R^2 = 1 - \frac{2.40}{6.00} = \mathbf{0.6000}, \quad s = \sqrt{\frac{2.40}{5-2}} = \sqrt{0.8} = \mathbf{0.8944}.$$

Two checks you should always run

$SS_{\text{reg}} = 6.00 - 2.40 = 3.60$, and independently $b_1^2 S_{xx} = 0.36 \times 10 = 3.60$. And

$r^2 = (6/\sqrt{10 \times 6})^2 = 36/60 = 0.6$. **For simple linear regression R^2 is the squared correlation.**

Drawn: the two sums of squares



Left, the dashed segments are the deviations from $\bar{y} = 4$: their squares add to 6.00. Right, the deviations from the fitted line: their squares add to 2.40. R^2 is the fraction of the left picture that the right picture removed — 60% of it.

The same numbers from the library

```
from sklearn.metrics import r2_score
print(r2_score(y, yhat)) # 1 - SSres/SStot
print(LinearRegression().fit(x[:, None], y).score(x[:, None], y))
```

```
SSreg = SStot - SSres = 3.60      b1^2 * Sxx = 3.60
R2 = 1 - SSres/SStot = 1 - 2.40/6.00 = 0.6000
r^2 = (Sxy/sqrt(Sxx*Syy))^2 = 0.6000
sklearn r2_score                = 0.6000
s = sqrt(SSres/(n-2))            = 0.8944
```

The same numbers from the library

```
from sklearn.metrics import r2_score
print(r2_score(y, yhat)) # 1 - SSres/SStot
print(LinearRegression().fit(x[:, None], y).score(x[:, None], y))
```

```
SSreg = SSStot - SSres = 3.60      b1^2 * Sxx = 3.60
R2 = 1 - SSres/SSStot = 1 - 2.40/6.00 = 0.6000
r^2 = (Sxy/sqrt(Sxx*Syy))^2 = 0.6000
sklearn r2_score                = 0.6000
s = sqrt(SSres/(n-2))            = 0.8944
```

Four routes to 0.6000: the definition, the regression sum of squares, the squared correlation, and `r2_score`. `r2_score` is **three lines of arithmetic, not a verdict**. The next four slides are about what those three lines cannot see.

Predict first #3

30 rows. **One** real predictor, with $y = 1 + 2x + \varepsilon$ and a noise sd of 2. Its R^2 is 0.2413. Now append columns of $\mathcal{N}(0, 1)$ **pure noise** — generated without ever looking at y — and refit, one column at a time, up to 27 of them.

What happens to R^2 ? Does it ever go down?

Predict first #3

30 rows. **One** real predictor, with $y = 1 + 2x + \varepsilon$ and a noise sd of 2. Its R^2 is 0.2413. Now append columns of $\mathcal{N}(0, 1)$ **pure noise** — generated without ever looking at y — and refit, one column at a time, up to 27 of them.

What happens to R^2 ? Does it ever go down?

extra noise columns	p	R2	adjusted R2
0	1	0.241296	+0.214199
5	6	0.402777	+0.246979
10	11	0.660344	+0.452777
15	16	0.743023	+0.426743
20	21	0.793909	+0.252920
27	28	0.940085	-0.737539

R2 decreased at any step? False (28 nested models)
 smallest single-step change +0.00000315

Predict first #3

30 rows. **One** real predictor, with $y = 1 + 2x + \varepsilon$ and a noise sd of 2. Its R^2 is 0.2413. Now append columns of $\mathcal{N}(0, 1)$ **pure noise** — generated without ever looking at y — and refit, one column at a time, up to 27 of them.

What happens to R^2 ? Does it ever go down?

extra noise columns	p	R2	adjusted R2
0	1	0.241296	+0.214199
5	6	0.402777	+0.246979
10	11	0.660344	+0.452777
15	16	0.743023	+0.426743
20	21	0.793909	+0.252920
27	28	0.940085	-0.737539

R2 decreased at any step? False (28 nested models)
smallest single-step change +0.00000315

0.2413 $\rightarrow$ 0.9401, and it never once fell

Not “rarely fell” — **never**, across all 27 steps, and the smallest step was $+4 \times 10^{-8}$, not -4×10^{-8} . Adding a column can only enlarge the space of fits available, and the old fit is still in it (set the new coefficient to zero). **So R^2 is a monotone function of model size and cannot be used to choose one.**

Adjusted R^2 : a correction, and its limits

$$R^2_{\text{adj}} = 1 - (1 - R^2) \frac{n - 1}{n - p - 1}$$

charges you p degrees of freedom. On the 30 rows it went $+0.2142 \rightarrow -0.7375$: it caught the fraud completely. Now the same experiment on 442 real patients:

```
load_diabetes: 442 rows, 10 real features
  noise cols    p      R2      adjusted R2    5-fold CV R2
         0      10  0.517748  +0.506559    +0.489155
        100     110  0.643486  +0.525008    +0.337363
        200     210  0.759013  +0.539935    -0.105104
        400     410  0.976075  +0.659652    -5.760536
```

Adjusted R^2 : a correction, and its limits

$$R^2_{\text{adj}} = 1 - (1 - R^2) \frac{n - 1}{n - p - 1}$$

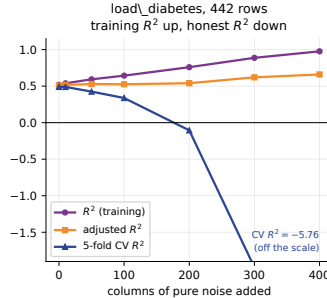
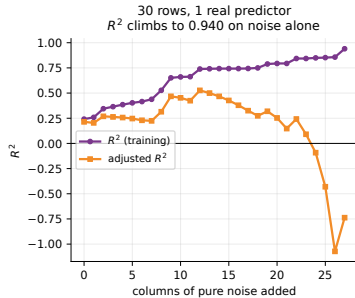
charges you p degrees of freedom. On the 30 rows it went $+0.2142 \rightarrow -0.7375$: it caught the fraud completely. Now the same experiment on 442 real patients:

```
load_diabetes: 442 rows, 10 real features
  noise cols    p      R2      adjusted R2    5-fold CV R2
         0      10  0.517748  +0.506559    +0.489155
        100     110  0.643486  +0.525008    +0.337363
        200     210  0.759013  +0.539935    -0.105104
        400     410  0.976075  +0.659652    -5.760536
```

Adjusted R^2 was still saying $+0.54$ when the truth was -8.83

With 410 junk columns on 442 rows, $R^2 = 0.9761$ and **adjusted R^2 went up, to 0.6597**. Cross-validation said -8.83 . **Adjusted R^2 is a penalty, not a test.** It is worth reporting and it is not worth trusting; the honest number is the one computed on rows the fit never saw.

Drawn: the two curves that disagree



Left, 30 rows: R^2 climbs to 0.908 on noise while adjusted R^2 collapses. Right, 442 rows: R^2 climbs, adjusted R^2 stays flat and complacent near 0.5, and cross-validated R^2 falls off the bottom of the chart. Only the third curve is measuring prediction.

Predict first #4

Three held-out points, $y_{\text{test}} = (1, 2, 3)$. Your model predicts $(3, 2, 1)$ — exactly backwards.

What is R^2 on those three points? Is it allowed to be below zero?

Predict first #4

Three held-out points, $y_{\text{test}} = (1, 2, 3)$. Your model predicts $(3, 2, 1)$ — exactly backwards.

What is R^2 on those three points? Is it allowed to be below zero?

```
y_test      [1.  2.  3.]
prediction   [3.  2.  1.]
mean of y_test 2.0000  SStot 2.0000  SSres 8.0000
R2 = 1 - 8.0/2.0 = -3.0000      sklearn -3.0000
R2 of the constant prediction 2.0: 0.0000
```

Predict first #4

Three held-out points, $y_{\text{test}} = (1, 2, 3)$. Your model predicts $(3, 2, 1)$ — exactly backwards.

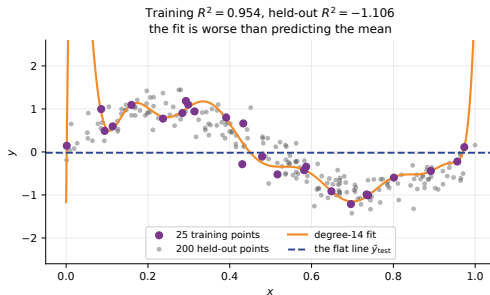
What is R^2 on those three points? Is it allowed to be below zero?

```
y_test      [1.  2.  3.]
prediction   [3.  2.  1.]
mean of y_test 2.0000  SStot 2.0000  SSres 8.0000
R2 = 1 - 8.0/2.0 = -3.0000      sklearn -3.0000
R2 of the constant prediction 2.0: 0.0000
```

$R^2 = -3$, and there is no lower bound at all

R^2 is only bounded below by 0 **on the data the intercept was fitted to**, where $\bar{y}$ is available to the model for free. On *held-out* rows $\bar{y}_{\text{test}}$ is a number the model never saw, so SS_{res} can exceed SS_{tot} without limit. **A negative held-out R^2 means: you would have done better predicting the mean and going home.**

Drawn: negative R^2 on real held-out data



A degree-14 polynomial through 25 training points: training $R^2 = 0.954$, held-out $R^2 = -1.106$.

The orange curve passes beautifully close to every purple point and leaves the data entirely between them. **The blue dashed line — predict the mean, always — beats it.**

What one number cannot tell you

Assumption	What breaks it, and how you see it
Linearity	the truth bends. Residuals versus fitted show a systematic arch or U.
Constant variance	the spread grows with $\hat{y}$. The residual plot fans out.
Normality	heavy tails or skew. The normal Q-Q plot bends away from the line at the ends.
Independence	time or space structure. Residuals plotted in order drift or oscillate.
No single point in charge	one high-leverage row. Delete it and the slope moves.

R^2 detects none of these

R^2 is a ratio of two sums. It cannot see the *order* of the residuals, their *shape*, or their *pattern against $\hat{y}$* . **Four datasets are about to have the same R^2 and four completely different problems.**

Anscombe's quartet, computed

Four datasets of eleven (x, y) pairs, published by F. J. Anscombe in 1973. You met them in L8; today we take the next step and read the residuals.

set	mean x	mean y	sd x	sd y	intercept	slope	R2	SSres
I	9.000	7.501	3.317	2.032	3.0001	0.5001	0.6665	13.7627
II	9.000	7.501	3.317	2.032	3.0009	0.5000	0.6662	13.7763
III	9.000	7.500	3.317	2.030	3.0025	0.4997	0.6663	13.7562
IV	9.000	7.501	3.317	2.031	3.0017	0.4999	0.6667	13.7425

Anscombe's quartet, computed

Four datasets of eleven (x, y) pairs, published by F. J. Anscombe in 1973. You met them in L8; today we take the next step and read the residuals.

set	mean x	mean y	sd x	sd y	intercept	slope	R2	SSres
I	9.000	7.501	3.317	2.032	3.0001	0.5001	0.6665	13.7627
II	9.000	7.501	3.317	2.032	3.0009	0.5000	0.6662	13.7763
III	9.000	7.500	3.317	2.030	3.0025	0.4997	0.6663	13.7562
IV	9.000	7.501	3.317	2.031	3.0017	0.4999	0.6667	13.7425

Same means to three decimals, same standard deviations, same fitted line $\hat{y} = 3.00 + 0.50x$, same $R^2 = 0.666$, same residual sum of squares to two decimals. **If you report the table above, you have told your reader that these four datasets are the same dataset.**

Anscombe's quartet, computed

Four datasets of eleven (x, y) pairs, published by F. J. Anscombe in 1973. You met them in L8; today we take the next step and read the residuals.

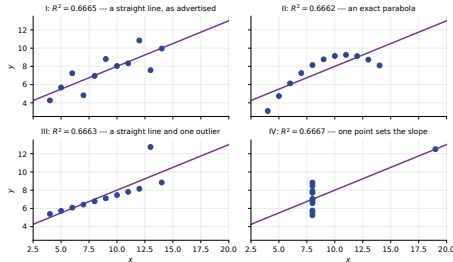
set	mean x	mean y	sd x	sd y	intercept	slope	R2	SSres
I	9.000	7.501	3.317	2.032	3.0001	0.5001	0.6665	13.7627
II	9.000	7.501	3.317	2.032	3.0009	0.5000	0.6662	13.7763
III	9.000	7.500	3.317	2.030	3.0025	0.4997	0.6663	13.7562
IV	9.000	7.501	3.317	2.031	3.0017	0.4999	0.6667	13.7425

Same means to three decimals, same standard deviations, same fitted line $\hat{y} = 3.00 + 0.50x$, same $R^2 = 0.666$, same residual sum of squares to two decimals. **If you report the table above, you have told your reader that these four datasets are the same dataset.**

They are not. Look at the next slide before you read the numbers again.

Drawn: the four datasets

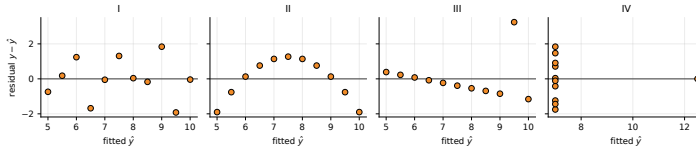
Anscombe's quartet: same $\bar{x}$, $\bar{y}$, sds, fitted line and R^2



I is a genuine noisy line. II is an exact parabola — the wrong model, not noisy data. III is a perfect straight line with one contaminated point. IV has ten points at $x = 8$ and one at $x = 19$: **the eleventh point alone determines the slope**, and there is no evidence of linearity anywhere.

The residual plot gives four different diagnoses

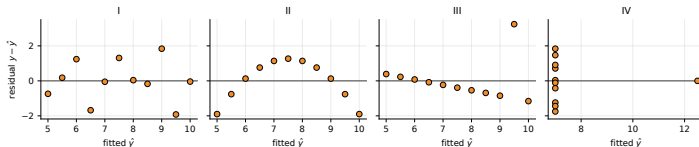
The residual plot the R^2 was hiding: same 0.666 four times



set	max e	e max/ e 2nd	curvature R2	Shapiro p	max leverage	slope w/o pt
I	1.921	1.04	0.0623	0.5456	0.3182	0.5028
II	1.901	1.00	1.0000	0.0929	0.3182	0.6267
III	3.241	2.80	0.0550	0.0016	0.3182	0.5770
IV	1.839	1.05	0.0000	0.7800	1.0000	undefined

The residual plot gives four different diagnoses

The residual plot the R^2 was hiding: same 0.666 four times



set	max e	e max/ e 2nd	curvature R2	Shapiro p	max leverage	slope w/o pt
I	1.921	1.04	0.0623	0.5456	0.3182	0.5028
II	1.901	1.00	1.0000	0.0929	0.3182	0.6267
III	3.241	2.80	0.0550	0.0016	0.3182	0.5770
IV	1.839	1.05	0.0000	0.7800	1.0000	undefined

II: a parabola fitted to the residuals explains **100.00%** of them — there is no noise left, only unmodelled curvature. **III:** the largest residual is $2.80\times$ the next, and Shapiro rejects normality at $p = 0.0016$. **IV:** leverage **1.0000**, and deleting that one row leaves the slope **undefined**. Same R^2 , four different failures.

Non-constant variance, measured

200 points, $y = 3 + 2x + x\epsilon/4$: the noise grows with x .

```
fit: intercept 2.8790  slope 2.0011  R2 0.9267
residual sd, lower half of x  1.0460
residual sd, upper half of x  1.8988  ratio 1.82
Breusch-Pagan LM = n*R2(e^2 on yhat) = 22.9065  chi2(1) p = 0.000002
Shapiro-Wilk on the residuals p = 0.031073
after log(y): Breusch-Pagan LM = 3.2426  p = 0.0717  sd ratio 0.89
```

Non-constant variance, measured

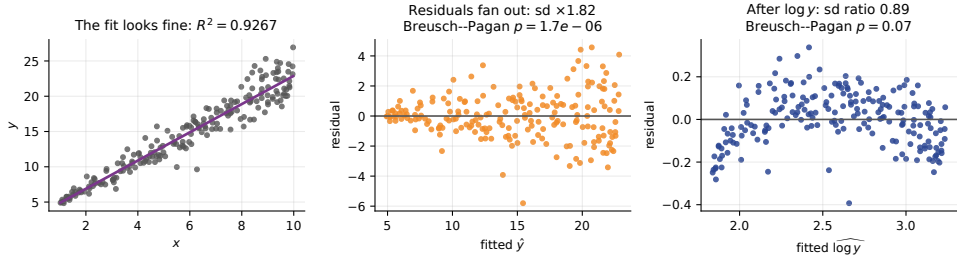
200 points, $y = 3 + 2x + x\epsilon/4$: the noise grows with x .

```
fit: intercept 2.8790  slope 2.0011  R2 0.9267
residual sd, lower half of x  1.0460
residual sd, upper half of x  1.8988  ratio 1.82
Breusch-Pagan LM = n*R2(e^2 on yhat) = 22.9065  chi2(1) p = 0.000002
Shapiro-Wilk on the residuals p = 0.031073
after log(y): Breusch-Pagan LM = 3.2426  p = 0.0717  sd ratio 0.89
```

$R^2 = 0.9267$ and the model is wrong anyway

The coefficients are still unbiased — least squares does not need constant variance for that. What breaks is everything *downstream*: standard errors, p -values, confidence intervals and prediction intervals are all computed as if σ were one number, and it is not. **log y takes it most of the way back: the test statistic falls from 22.91 to 3.24, p from 2×10^{-6} to 0.072 — an improvement, not a cure.**

Drawn: the fan, and what a transform does to it



The left panel is the one you would put in a report. The middle panel is the one that tells you the truth — the residual cloud is a wedge, wide on the right. The right panel is the same data after log y: **a shapeless band, which is what a residual plot should look like.**

The normal Q–Q plot, and how to read it

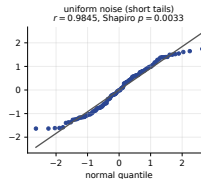
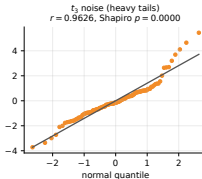
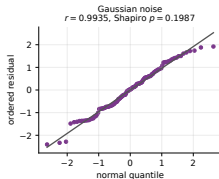
Sort the residuals; plot them against the normal quantiles they would occupy if they were Gaussian. Straight line, Gaussian residuals.

residual model	corr(ordered resid, normal quantile)	Shapiro p	kurtosis
Gaussian noise	0.9935	0.198668	-0.5004
t(3) noise	0.9626	0.000009	2.7920
uniform noise	0.9845	0.003336	-1.0901

The normal Q–Q plot, and how to read it

Sort the residuals; plot them against the normal quantiles they would occupy if they were Gaussian. Straight line, Gaussian residuals.

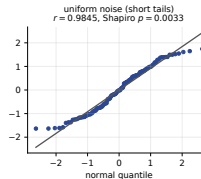
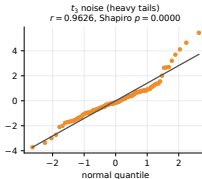
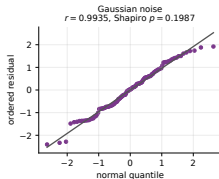
residual model	corr(ordered resid, normal quantile)	Shapiro p	kurtosis
Gaussian noise	0.9935	0.198668	-0.5004
t(3) noise	0.9626	0.000009	2.7920
uniform noise	0.9845	0.003336	-1.0901



The normal Q–Q plot, and how to read it

Sort the residuals; plot them against the normal quantiles they would occupy if they were Gaussian. Straight line, Gaussian residuals.

residual model	corr(ordered resid, normal quantile)	Shapiro p	kurtosis
Gaussian noise	0.9935	0.198668	-0.5004
t(3) noise	0.9626	0.000009	2.7920
uniform noise	0.9845	0.003336	-1.0901



All three correlations are above 0.97 — the eyeball is far more discriminating here than the summary statistic. t_3 bends away from the line at both ends (heavy tails); uniform bends *towards* it (short tails). Neither is fatal for the coefficients; both invalidate the p -values.

Over-fitting, defined by two error curves

A model **over-fits** when it is learning the noise in the training rows — structure that will not repeat.

The signature

Training error keeps falling. Error on rows the model has never seen **stops falling and turns up**. The gap between the two is the amount of noise memorised.

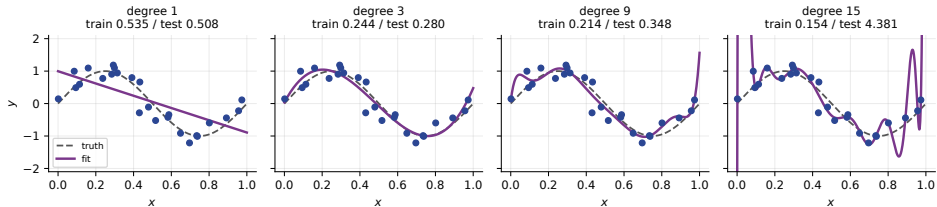
Why it must happen

A degree- d polynomial through n points has $d + 1$ free parameters. At $d + 1 = n$ it passes through *every* point exactly, noise included, and $R^2 = 1$. **Perfect training fit is a symptom, not an achievement.**

The running example: 25 points from $y = \sin 2\pi x + \varepsilon$ with $\sigma = 0.25$. **We know the truth and the noise floor, which is why this example can be graded.**

Drawn: the same 25 points, four models

25 points from $y = \sin 2\pi x + \varepsilon$: the same data, four models



Degree 1 cannot bend at all — **under-fitting**, and both errors are large. Degree 3 tracks the truth. Degree 9 has started to ripple. Degree 15 passes near every point and **leaves the dashed truth entirely**: train 0.154, test 4.381.

Predict first #5

Fit polynomials of degree 0 to 20 to those 25 points. Score each on the training rows and on 2000 fresh rows from the same generator.

At which degree does the test error stop improving? And how bad does degree 20 get?

Predict first #5

Fit polynomials of degree 0 to 20 to those 25 points. Score each on the training rows and on 2000 fresh rows from the same generator.

At which degree does the test error stop improving? And how bad does degree 20 get?

degree	params	train RMSE	test RMSE
1	2	0.5347	0.5084
3	4	0.2441	0.2795
5	6	0.2273	0.2828
9	10	0.2143	0.3484
12	13	0.1843	0.5966
15	16	0.1540	4.3811
20	21	0.1466	153.1554

train RMSE strictly non-increasing? True largest increase -3.29e-05
test minimum at degree 3, RMSE 0.2795 (noise floor 0.2500)

Predict first #5

Fit polynomials of degree 0 to 20 to those 25 points. Score each on the training rows and on 2000 fresh rows from the same generator.

At which degree does the test error stop improving? And how bad does degree 20 get?

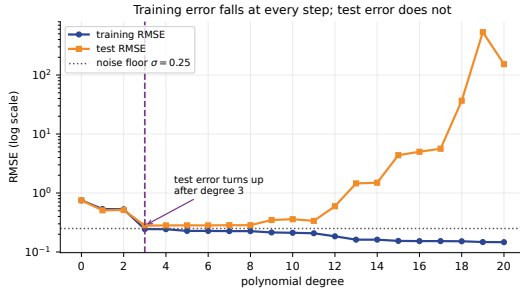
degree	params	train RMSE	test RMSE
1	2	0.5347	0.5084
3	4	0.2441	0.2795
5	6	0.2273	0.2828
9	10	0.2143	0.3484
12	13	0.1843	0.5966
15	16	0.1540	4.3811
20	21	0.1466	153.1554

train RMSE strictly non-increasing? True largest increase -3.29e-05
test minimum at degree 3, RMSE 0.2795 (noise floor 0.2500)

The turn is at degree 3, and degree 20 is **548×** worse

Train RMSE fell at *every one* of the twenty steps — verified, not asserted — from 0.7507 to 0.1466. Test RMSE bottomed at 0.2795, a whisker above the noise floor of 0.2500, and then went to 153. **Between degree 3 and degree 20 the training score improved by 40% and the model became unusable.**

Drawn: the two curves that define over-fitting



Log scale on the vertical axis, or the right-hand end would not fit on the page. **The blue curve is monotone by construction** — a degree- $(d+1)$ model contains every degree- d model. The orange curve owes you nothing, and after degree 3 it stops paying.

More rows move the turning point

The same generator, the same degrees, only n_{train} changes.

n_train	best degree	test RMSE at best	test RMSE at degree 20
15	3	0.2959	13292.3268
25	5	0.3102	2867.4761
50	6	0.2634	0.3256
200	5	0.2547	0.2656
1000	7	0.2536	0.2572

More rows move the turning point

The same generator, the same degrees, only n_{train} changes.

n_train	best degree	test RMSE at best	test RMSE at degree 20
15	3	0.2959	13292.3268
25	5	0.3102	2867.4761
50	6	0.2634	0.3256
200	5	0.2547	0.2656
1000	7	0.2536	0.2572

At $n = 1000$, degree 20 costs almost nothing

0.2572 against a best of 0.2536 and a noise floor of 0.2500. The *same* twenty-one-parameter model that was catastrophic on 15 rows is harmless on 1000. **Over-fitting is not a property of the model. It is a property of the model and the amount of data you have** — which is why “how complex a model may I use?” has no answer until you say n .

And it happens on real data, not just sine waves

load_diabetes, 442 patients, split 75/25. Add polynomial interactions of the ten features.

model	train R2	5-fold CV R2	held-out R2	params
linear (10 features)	+0.5554	+0.5224	+0.3594	10
degree-2 polynomial	+0.6469	+0.3827	+0.2440	65
degree-3 polynomial	+0.9179	-1212.1828	-22.0428	285
predicting the training mean for every patient: held-out R2 -0.0001				

And it happens on real data, not just sine waves

load_diabetes, 442 patients, split 75/25. Add polynomial interactions of the ten features.

model	train R2	5-fold CV R2	held-out R2	params
linear (10 features)	+0.5554	+0.5224	+0.3594	10
degree-2 polynomial	+0.6469	+0.3827	+0.2440	65
degree-3 polynomial	+0.9179	-1212.1828	-22.0428	285
predicting the training mean for every patient: held-out R2 -0.0001				

Training R^2 of 0.92, held-out R^2 of -22

285 parameters and 331 training rows. The training column goes **up** at every row of the table and the held-out column goes **down** at every row. **Reading only the training column, you would ship the worst of the three.** And the last line is the yardstick: even predicting a constant beats -22.

The idea, in three sentences

k -fold cross-validation — the mechanism from Unit IV

Split the training rows into k equal parts. For each part in turn: fit on the other $k - 1$ parts, score on the held-out part. Average the k scores.

What it buys

Every row is used for testing exactly once and for training $k - 1$ times. **You get a held-out estimate without spending a held-out set**, and you get k of them, so you get a spread too.

What it costs

k model fits instead of one. And **LOOCV**, the case $k = n$, costs n fits — but has no split lottery at all, because there is only one way to leave one out.

The test set is **not** part of this. It is opened once, at the end, to report a number — never to choose a degree.

The detector, run on the over-fitting example

```
pipe = make_pipeline(PolynomialFeatures(d), LinearRegression())
cv = -cross_val_score(pipe, x[:, None], y, cv=KFold(5, shuffle=True,
               random_state=0), scoring="neg_root_mean_squared_error")
```

degree	train RMSE	5-fold CV RMSE	LOOCV RMSE	test RMSE
1	0.5347	0.6319	0.5895	0.5084
3	0.2441	0.3239	0.2799	0.2795
5	0.2273	0.6711	0.2912	0.2828
9	0.2143	32.7952	0.4516	0.3484
15	0.1540	23379.8414	49.8944	4.3811
train RMSE picks degree 15 (the largest offered)				5-fold CV picks degree 3
LOOCV picks degree 3				the test set says 3

The detector, run on the over-fitting example

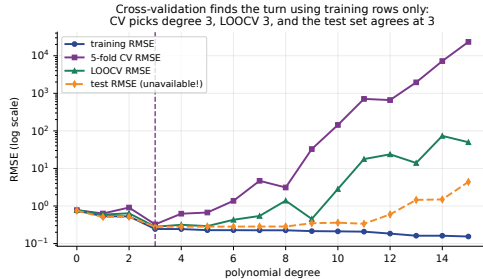
```
pipe = make_pipeline(PolynomialFeatures(d), LinearRegression())
cv = -cross_val_score(pipe, x[:, None], y, cv=KFold(5, shuffle=True,
               random_state=0), scoring="neg_root_mean_squared_error")
```

degree	train RMSE	5-fold CV RMSE	LOOCV RMSE	test RMSE
1	0.5347	0.6319	0.5895	0.5084
3	0.2441	0.3239	0.2799	0.2795
5	0.2273	0.6711	0.2912	0.2828
9	0.2143	32.7952	0.4516	0.3484
15	0.1540	23379.8414	49.8944	4.3811
train RMSE picks degree 15 (the largest offered)				5-fold CV picks degree 3
LOOCV picks degree 3				the test set says 3

Three of the four columns agree, and it is the wrong one that differs

Train RMSE picks the biggest model on offer, as it always will. **5-fold CV and LOOCV both pick degree 3 — the same answer the test set gives — using nothing but the 25 training rows.** That is the whole argument for cross-validation in one table.

Drawn: four curves, one of them unavailable



The orange dashed curve is the test error — drawn here only because this is a demonstration; **in a real project you may not look at it**. The purple and green curves reproduce its minimum without it. LOOCV tracks the test curve more closely at small degree; 5-fold is noisier and 3× cheaper here, 5× against 25 fits.

Why not just hold out one validation set?

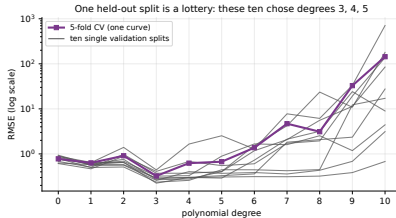
Split the same 25 rows 60/40, ten different ways, and let each split choose the degree.

```
ten different single validation splits of the same 25 rows
degree chosen by each split: [3, 3, 3, 3, 4, 5, 3, 3, 3, 3]
chosen degrees range from 3 to 5
spread of the estimated RMSE at degree 3: 0.2243 to 0.4415
5-fold CV RMSE at degree 3 (one number, no split lottery): 0.3239
```


Why not just hold out one validation set?

Split the same 25 rows 60/40, ten different ways, and let each split choose the degree.

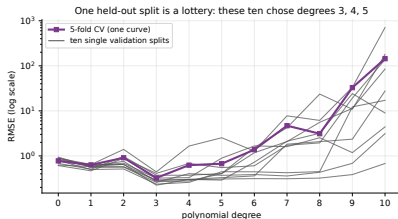
```
ten different single validation splits of the same 25 rows
degree chosen by each split: [3, 3, 3, 3, 4, 5, 3, 3, 3, 3]
chosen degrees range from 3 to 5
spread of the estimated RMSE at degree 3: 0.2243 to 0.4415
5-fold CV RMSE at degree 3 (one number, no split lottery): 0.3239
```



Why not just hold out one validation set?

Split the same 25 rows 60/40, ten different ways, and let each split choose the degree.

```
ten different single validation splits of the same 25 rows
degree chosen by each split: [3, 3, 3, 3, 4, 5, 3, 3, 3, 3]
chosen degrees range from 3 to 5
spread of the estimated RMSE at degree 3: 0.2243 to 0.4415
5-fold CV RMSE at degree 3 (one number, no split lottery): 0.3239
```



Seven of the ten splits chose degree 3 and three did not — and the *estimate* of the error at degree 3 ranged from 0.2243 to 0.4415, a **factor of two**, from nothing but which rows landed in which half. *k*-fold averages that lottery away and uses every row.

Choosing k , and the rule that outranks it

	Bias of the estimate	Variance / cost
$k = 2$	pessimistic: each fit sees half the data	cheap, but noisy
$k = 5$ or 10	small; the usual choice	moderate, 5–10 fits
$k = n$ (LOOCV)	almost none: each fit sees $n - 1$ rows	n fits; the folds overlap heavily

The rule that matters more than k

Everything the model learns must be refitted inside every fold. A degree, a scaler, a feature selection, a threshold. If a choice was made using all the rows and then cross-validated, the score is not a held-out score — as the splitting and validation lecture measured, and as Unit VI will measure again.

Pipeline exists precisely so that this is the easy thing to do.

Summary — twelve lines

1. “Closest” is a choice. On five points the direct slope is 0.6000, the inverse slope 1.0000, the orthogonal slope 0.7208; the ratio of the first two is $r^2 = 0.6$. Each wins only its own criterion.
2. **Assume $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$ i.i.d., take logs, and the log-likelihood is const $- \text{SSE}/(2\sigma^2)$. Maximising it is exactly minimising the SSE.** Least squares is the Gaussian MLE.
3. σ^2 does not move the estimate: the same $b_1 = 0.60000$ maximised the likelihood at $\sigma^2 = 0.48$ and at $\sigma^2 = 1$.
4. $\hat{\sigma}_{\text{ML}}^2 = \text{SSE}/n$ is **biased downwards** by $(n-2)/n$. Over 20,000 datasets with $\sigma^2 = 4$ and $n = 10$ it averaged 3.2117 against a predicted 3.2000, and was too small in 73.4% of runs. Use $s^2 = \text{SSE}/(n-2)$.
5. Change the noise to Laplace and the MLE becomes least *absolute* deviations. Each estimator won its own criterion.
6. $R^2 = 1 - \text{SS}_{\text{res}}/\text{SS}_{\text{tot}} = 1 - 2.40/6.00 = 0.6000$ on the five points, equal to r^2 and to $b_1^2 S_{xx}/\text{SS}_{\text{tot}}$.
7. R^2 **never falls when you add a column**. 27 pure-noise columns on 30 rows took it from 0.2413 to 0.9401, never once decreasing.
8. Adjusted R^2 caught that (-0.7375) but on 442 rows with 410 junk columns it read $+0.6597$ while cross-validation read -5.7605 . A penalty, not a test.
9. Held-out R^2 has **no lower bound**: -3 on three points by hand, -1.106 for a degree-14 fit, -22.04 on diabetes.
10. Anscombe’s four datasets share every summary statistic and $R^2 = 0.666$. The residual plot separates them: curvature $R^2 = 1.0000$ for II, largest residual $2.80\times$ the next for III, leverage 1.0000 for IV.
11. Over-fitting: train RMSE fell at all 20 steps, test RMSE turned up after **degree 3** and reached 153 at degree 20 — $548\times$ worse. At $n = 1000$ the same degree 20 costs 0.0036.
12. **5-fold CV and LOOCV both picked degree 3, the same answer as the test set, using only the training rows.** One held-out split picked 3, 4 or 5 depending on the seed.

Before the next lecture

Read

notes.pdf — the maximum likelihood derivation written out line by line, the proof that $\mathbb{E}[\text{SSE}] = (n - 2)\sigma^2$, and **10 practice questions with full worked solutions**.

Do

Questions 2, 4 and 7. Question 2 is the derivation from a blank page; question 4 is R^2 by hand; question 7 is a debugging story you will meet for real.

Next: logistic regression

The response becomes a class label.

Carry the method, not the formula:

write the likelihood, take logs, maximise.

Squares will not appear — because the noise is no longer Gaussian.

And a habit to start now

Never report an R^2 without a residual plot beside it, and never report a training score as if it were a result.

Materials: <https://github.com/tali7c/Applied-Machine-Learning>

Sources: James et al., *An Introduction to Statistical Learning with Python*, Ch. 3–6.; Bishop, *Pattern Recognition and Machine Learning*, Ch. 3.; Deisenroth, Faisal and Ong, *Mathematics for Machine Learning*, Ch. 9.; Müller and Guido, *Introduction to Machine Learning with Python*, §2.3.